# Tema 4: Funciones

## Fundamentos de Programación

Francisco J. Rodríguez Díaz
(fjrodriguez@decsai.ugr.es)
Departamento de Ciencias de la Computación e I.A.

Universidad de Granada

# Tabla de Contenidos I

1 Funciones

# Contenido del tema

1 Funciones

# Módulos

## Módulos

- Los lenguajes de programación proporcionan mecanismos para englobar un conjunto de sentencias en un paquete o **módulo**: funciones y clases.
- El objetivo es empaquetar un conjunto de instrucciones que resuelven **una tarea concreta**.
- Ya los hemos usado, por ejemplo, **sqrt** y **pow** son funciones implementadas en las biblioteca **cmath**.

# Funciones (Motivación I)

## Motivación

```
1  int main() {
2    double lado1_A, lado2_A, lado1_B, lado2_B,
3      hipotenusa_A, hipotenusa_B,
4      radicando_A, radicando_B;
5
6    <Asignación de valores a los lados>;
7
8    radicando_A = lado1_A * lado1A + lado2_A * lado2_A;
9    hipotenusa_A = sqrt(radicando_A);
10
11   radicando_B = lado1_B * lado1_B + lado2_B * lado2_B;
12   hipotenusa_B = sqrt(radicando_B);
13
14 }
```

# Funciones (Motivación II)

## Solución

```
1  int main() {
2    double lado1_A, lado2_A, lado1_B, lado2_B,
3      hipotenusa_A, hipotenusa_B,
4      radicando_A, radicando_B;
5
6    <Asignación de valores a los lados>;
7
8    hipotenusa_A = Hipotenusa(lado1_A, lado2_A);
9    hipotenusa_B = Hipotenusa(lado1_B, lado2_B);
10
11 }
```

# Funciones (Definición)

## Definición

```
<tipo> <nombre-función> (<parámetros formales>) {
  <sentencias>;
  return <expresión>;
}
```

## Definición de funciones

- Definición después de la inclusión de bibliotecas y antes del *main*.
- Diremos que *<tipo> <nombre-función> (<parámetros formales>)* es la **cabecera (header) de la función**
- El cuerpo de la función debe contener *return <expresión>*, donde expresión debe ser del mismo tipo (o compatible) especificado en la cabecera con *<tipo>*.

# Funciones (Parámetros formales y actuales)

## Parámetros formales

- Los **parámetros formales** son los que se especifican en la cabecera.
- Al declararlo hay que **especificar su tipo**. Si hay más de uno, se separan con comas.
- Los parámetros formales **solo se conocen dentro del cuerpo** de la función.

# Funciones (Parámetros formales y actuales)

## Parámetros formales

- Los **parámetros formales** son los que se especifican en la cabecera.
- Al declararlo hay que **especificar su tipo**. Si hay más de uno, se separan con comas.
- Los parámetros formales **solo se conocen dentro del cuerpo** de la función.

## Parámetros actuales

- Los **parámetros actuales** son las expresiones pasadas como argumento en las llamadas a una función.
- La llamada a la función tiene este formato: *<nombre-función> (<parámetros actuales>)*.
- En la llamada no se especifica el tipo. Si hay más de un parámetro, se separan con comas.

# Funciones (Ejemplo)

## Definición

```
1  double Cuadrado(double entrada) {  // entrada: parámetro formal
2    return entrada * entrada;
3  }
4
5  int main() {
6    double resultado, dato;
7    dato = 4;
8
9    resultado = Cuadrado(dato);
10       // dato: parámetro actual
11       // Cuadrado(dato) llamada a la función, expresión tipo real
12
13   cout << "El cuadrado de " << dato << " es " << resultado << endl;
14 }
```

# Funciones (Flujo de control I)

## Al llamar a la función

1. Se asigna el valor de la expresión de cada *parámetro actual* a su correspondiente *parámetro formal*: *<parámetro formal>* = *<parámetro actual>*
2. Empiezan a ejecutarse las sentencias del cuerpo de la función.
3. Cuando se llega a una sentencia *return <expresión>* termina la ejecución der la función, devolviendo el resultado de evaluar *<expresión>*
4. El flujo de control prosigue por donde se detuvo al realizar la invocación a la función.

# Funciones (Flujo de control II)

## Definición

```
1  double Cuadrado(double entrada) {  // 3. Se asigna a entrada el valor 4
2    return entrada * entrada; // 4. Se evalúa la expresión, resultado 16
3  }
4
5  int main() {
6    double resultado, dato;
7    dato = 4;
8
9    resultado = Cuadrado(dato);
10       // 1. Se evalúa dato, valor 4
11       // 2. Se invoca a la función
12       // 5. Se asigna a resultado el valor devuelto por la función.
13
14   cout << "El cuadrado de " << dato << " es " << resultado << endl;
15 }
```

# Funciones (Correspondencia parámetros actuales - formales)

## Correspondencia

Por orden de aparición, de izquierda a derecha

```
1  double Resta(double operando1, double operando2)
2    return operando1 - operando2;
3  }
4
5  int main() {
6    double un_valor = 5.0;
7    double otro_valor = 4.0;
8    double resta;
9
10   resta = Resta(un_valor, otro_valor);
11     // 1. operando1 toma valor 5.0, operando2 4.0
12     // 2. resta vale 1.0
13   resta = Resta(un_valor*2, otro_valor);
14     // 1. operando1 toma valor 10.0, operando2 4.0
15     // 2. resta vale 6.0
16 }
```

# Funciones (Otro ejemplo)

```
1  double Cuadrado(double entrada) {
2    return entrada * entrada;
3  }
4
5  double Hipotenusa(double un_lado, double otro_lado) {
6    return sqrt(Cuadrado(un_lado) + Cuadrado(otro_lado));
7  }
8
9  int main() {
10   double lado1_A, lado2_A, lado1_B, lado2_B,
11     hipotenusa_A, hipotenusa_B;
12   .....
13   hipotenusa_A = Hipotenusa(lado1_A, lado2_A);
14   hipotenusa_B = Hipotenusa(lado1_B, lado2_B);
15   .....
16 }
```

# Procedimientos (Funciones void)

## Procedimientos

Si una función no tiene sentencia return, se denomina **procedimiento** y en la cabecera de la función se incluye *void* como <*tipo*>

```
1  void MostrarMensaje(string cadena) {
2    cout << cadena << endl;
3  }
4
5  int main() {
6    string mi_mensaje;
7    mi_mensaje = "Esta función no tiene sentencia return";
8
9    MostrarMensaje(mi_mensaje);
10 }
```

# Funciones (Ámbito de las variables)

## Ámbito de las variables

- El **ámbito (scope)** de una variable o constante es el conjunto de todos aquellos sitios donde se puede acceder a su valor.
- Dentro de una función podemos declarar constantes y variables. Su valor solo se conocerá dentro del cuerpo de la función, se les denomina **datos locales**
  - Variable en la que se almacena el valor devuelto por la función.
  - Almacenar datos temporales requeridos para los cómputos de la función.
- Los parámetros formales actúan como datos locales.

# Funciones (Ámbito de las variables)

```
long long Factorial(int n) {
  int i;
  long long resultado = 1;

  for (i=2; i <= n; i++) {
    resultado = resultado * i;
  }

  return resultado;
}

int main() {
  int dato = 3;
  int salida;

  salida = Factorial(dato);
}
```

# Funciones (Paso de parámetros por valor)

## Parámetros por valor

- Las modificaciones del parámetro formal no afectan al valor del parámetro actual.
- Trabajamos con una copia del valor correspondiente al valor actual.

```
1  double Cuadrado(double entrada) {
2    entrada = entrada * entrada;
3    return entrada;
4  }
5
6  int main() {
7    double resultado, valor = 4.0;
8
9    resultado = Cuadrado (valor);
10   // después de llamar a la función:
11   // 1. resultado tiene valor 16.0
12   // 2. valor sigue siendo 4.0
13   .....
14 }
```

# Funciones (Paso de parámetros por referencia I)

## Parámetros por referencia

- El paso de parámetros por referencia hace que **el parámetro formal sea un alias de la variable que se pasa como parámetro, permitiendo modificar su valor dentro de la función**.
- Se indica anteponiendo el símbolo **&** al nombre del parámetro formal que se quiere pasar por referencia.
- Permite que una función **pueda devolver más de un valor**.

## Paso por referencia

```
int MiFunción(int dato1, int &dato2);
```

# Funciones (Paso de parámetros por referencia II)

## Paso por referencia

```
1  void Intercambiar(int &dato1, int &dato2) {
2    int temporal;
3    temporal = dato1;
4    dato1 = dato2;
5    dato2 = temporal;
6  }
7
8  int main() {
9    int un_valor = 5;
10   int otro_valor = 10;
11   Intercambiar(un_valor,otro_valor);
12   cout << "un_valor es " << un_valor << " y otro_valor es "
13     <<  otro_valor << endl;
14   // "un_valor es 10 y otro_valor es 5"
15
16 }
```

# Variables Globales

## Variables globales

Son variables definidas fuera de las funciones.
Se pueden usar en las funciones sin pasar de parámetro.

```
1  int cont;
2
3  int contador() {
4     cont += 1;
5     return cont;
6  }
```

```
1  void main() {
2     cout <<"Contador:";
3     cout <<contador() <<endl;
4     cout <<"Contador:";
5     count <<contador() <<endl;
6  }
```

```
Contador: 1
Contador: 2
```

# Variables Globales

## Problema

- No hay control de la variable.
- Se puede cambiar en todo el código.
  - A menudo por error.

```
...
suma = 0;

for (contador = 1; contador = 5, contador++) {
    suma += fabs(contador);
}
```

# Variables Globales

## Problema

- No hay control de la variable.
- Se puede cambiar en todo el código.
  - A menudo por error.

```
...
suma = 0;

for (contador = 1; contador = 5, contador++) {
    suma += fabs(contador);
}
```

Aviso: Evitar el uso de variables globales

# Estructuras y paso de Parámetros

## Paso por parámetro

Se pueden pasar parámetros de nuestros tipos por parámetro o como resultado de un return.

## Paso por parámetro

```
1  Punto suma(Punto p1, Punto p2) {
2      Punto p;
3      p.x = p1.x+p2.x;
4      p.y = p1.y+p2.y;
5      return p;
6  }
```

# Vectores como parámetros

```
<tipo> <identificador_funcion> (const <tipo>
  <identificador_vector>[], ...)
}
```

```
1  int minimo(const int valores[], int utiles) {
2    int menor = valores[0];
3    for (int i = 0; i < utiles; i++) {
4      if (valores[i] < menor) {
5        menor = valores[i];
6      }
7    }
8    return menor;
9  }
```

# Vectores como parámetros

```
<tipo> <identificador_funcion> (<tipo>
  <identificador_vector>[], ...)
}
```

```
1  void pares_impares(const int original[], int util_orig, int pares[],
2    int &util_par, int impares[], int &util_impar) {
3    util_par = util_impar = 0;
4    for (int i = 0; i < util_orig; i++) {
5      if (original[i] % 2 == 0) {
6        pares[util_par] = original[i];
7        util_par++;
8      }
9      else {
10       impares[util_impar] = original[i];
11       util_impar++;
12     }
13   }
14 }
```

# Vectores como parámetros

```
1   int main() {
2     const int MAX_SIZE = 100;
3     int datos[MAX_SIZE] = {1,4,5,6,2,8,9,10,11,13,14};
4     int utiles = 11;
5     int par[MAX_SIZE];
6     int impar[MAX_SIZE];
7     int utilp, utilimp;
8
9     pares_impares(datos, utiles, par, utilp, impar, utilimp);
10
11    for (int i = 0; i < utilp; i++) {
12      cout << par[i] << endl;
13    }
14    cout << endl;
15
16    for (int i = 0; i < utilimp; i++) {
17      cout << impar[i] << endl;
18    }
19    cout << endl;
20  }
```

# Matrices como parámetros

```
<tipo> <identificador_funcion> ( const <tipo>
  <identificador_matriz>[][COLUMNAS], ...)
}

<tipo> <identificador_funcion> ( <tipo>
  <identificador_matriz>[][COLUMNAS], ...)
}
```

# Matrices como parámetros

```
1   void LeerMatriz(int m[][MAXCOL], int &filas, int &columnas) {
2     cout << ¿Cuantas filas tiene la matriz? << endl;
3     cin >> filas;
4     cout << ¿Cuantas columnas tiene la matriz? << endl;
5     cin >> columnas;
6
7     for (int i = 0; i < filas; i++) {
8       for (int j = 0; j < columnas; j++) {
9         cout << "Introduce la posición [" << i << ", " << j << "]" << endl;
10        cin >> m[i][j];
11      }
12    }
13  }
```

# Matrices como parámetros

```
1   void MostrarMatriz(const int m[][MAXCOL], int filas, int columnas) {
2
3     for (int i = 0; i < filas; i++) {
4       for (int j = 0; j < columnas; j++) {
5         cout << m[i][j] << " ";
6       }
7       cout << endl;
8     }
9   }
10  ...
11  const int MAXCOL = 100;
12  const int MAXFILAS = 100;
13  ...
14  int main() {
15    int mi_matriz[MAXFILAS][MAXCOL];
16    int filas_utiles, columnas_utiles;
17
18    LeerMatriz(mi_matriz, filas_utiles, columnas_utiles);
19    MostrarMatriz(mi_matriz,filas_utiles, columnas_utiles);
20  }
```